AI Agents for Social Science Research

Workshop, Sociological Institute (SUZ), University of Zurich

Tuesday, 8 September 2026, 9:00-17:00

Paul Götz and Nico Mutzner

Run of Show

Time	Block
9:00	Part 1: LLMs and coding agents (type-along)
10:00	Part 2: one research workflow, end to end
12:15	Lunch
13:15	Part 3a: demos — a research hub, then a dataset
14:30	Part 3b: one literature workflow, live
14:50	Part 3c: hands-on in tracks (break ~15:40)
16:30	Part 4: open discussion, then feedback

Before We Start: Is Your Session Running?

Handout, Part 0 takes three steps and five minutes:

1. Clone or download the repository (`suz-ai-agent-workshop`)
2. Copy `walkthrough/project/` to a working folder (e.g. `swiss-rents`)
3. Start the agent **inside** that folder
    - CLI: `cd` into it, run `claude`
    - Desktop: Code tab, Local, Select folder

The working-directory contract: `.claude/` and `CLAUDE.md` load only if the session starts inside the project folder.

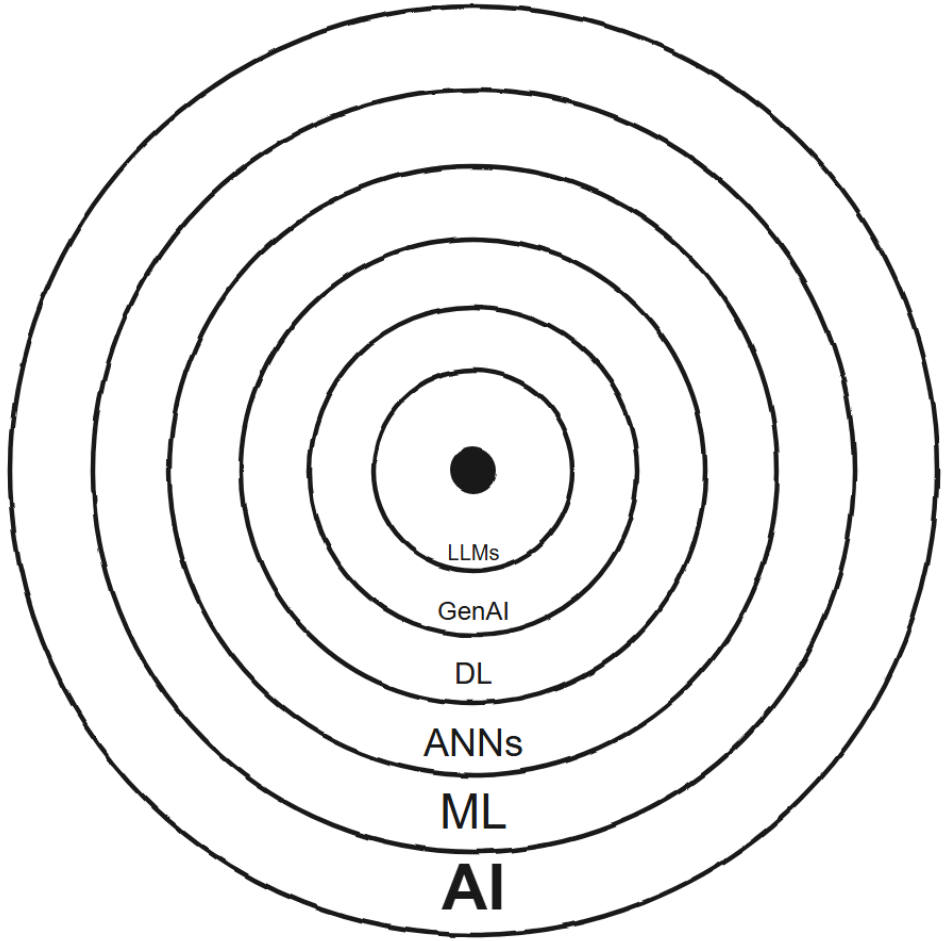
Part 1: LLMs and Coding Agents

9:00-10:00

- What a language model is and how it is built
- Which model to use, and how hard to let it think
- The two surfaces: chat + Projects, and Claude Code
- What turns a model into an agent: tools, the loop, permissions

Theory from the front, then four short prompts you run yourself. The prompts appear in the handout under "Part 1."

The AI Landscape: LLMs Are Just a Small Corner



How LLMs Are Built, 1: Pre-Training

- A neural network trained on very large text corpora
- One objective: predict the next token, billions of times
- The result is a base model: broad knowledge of language and the world, no notion of being helpful
- Everything the model "knows" is a statistical pattern in that training data, with a cutoff date

How LLMs Are Built, 2: Post-Training

- The base model is then tuned to be an assistant
- Instruction tuning: examples of questions paired with good answers
- Human feedback: people rank outputs; the model learns the ranking
- This step sets tone, refusals, honesty, formatting: the "personality" you interact with
- **Whose feedback?** The raters' instructions, judgments and demographics are baked into the model and are almost never published
- **This labour has a documented history of exploitation**: the workers who made ChatGPT safe to release were paid **$1.32-$2 an hour** to read the worst text on the internet

How LLMs Are Built, 3: Reasoning Models

- Newer models can "think before answering": they generate internal working-out before the final response
- Trained to plan, check, and revise multi-step work
- Cost: more time and more tokens per answer
- Benefit: large gains on math, code, and multi-step analysis, exactly the agent workload

See One From the Inside

https://bbycroft.net/llm

A working GPT model, drawn in 3D, with your own text running through it: embedding, attention, the feed-forward layers, one token coming out the far end.

Every number in the animation is a real number from a real forward pass.

There is no other step. That is the whole machine.

Non-Determinism, and What It Means for Research

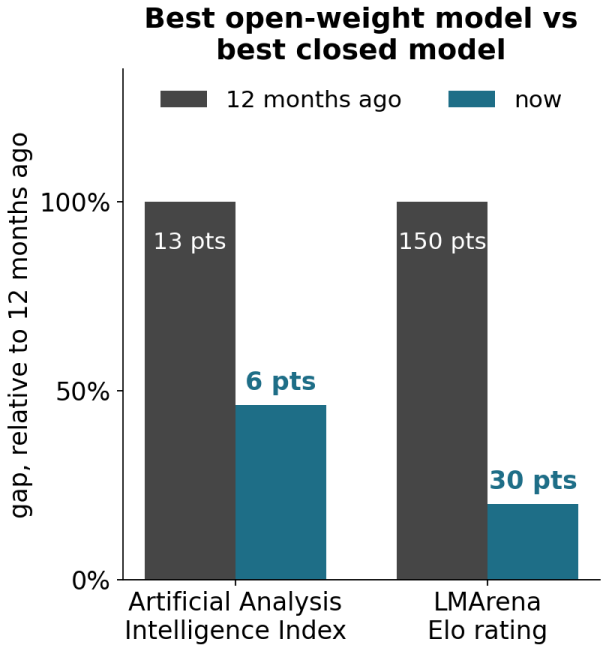
- The model outputs a probability distribution over next tokens; sampling picks from it
- Same prompt, different runs, different answers, by design
- **Caveat: that is a setting.** Temperature 0 and a fixed seed make sampling deterministic in principle, but you cannot set either in the chat products or in the tools we use today
- **And even then it is best-effort.** Providers batch requests, update models, and retire them; bit-identical output is not promised
- An LLM run is not reproducible the way a script is
- The consequence for research: put results in scripts and data files, not in the transcript

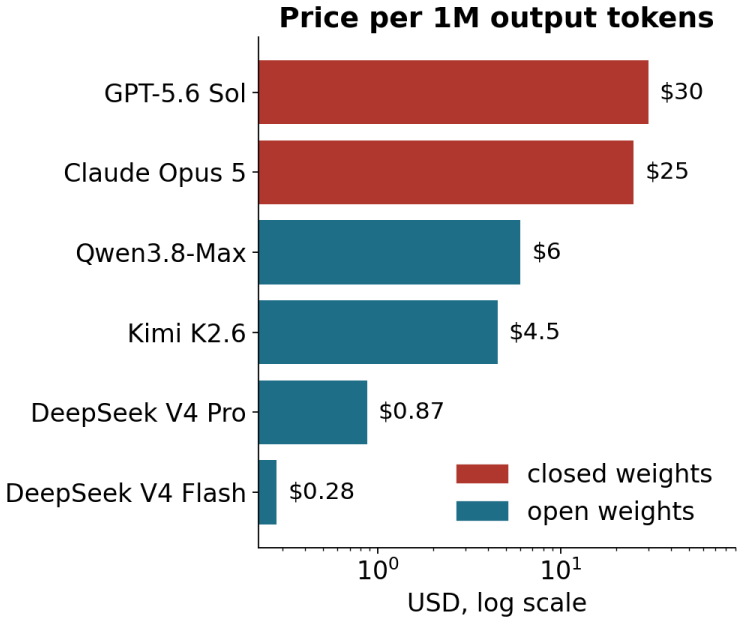
Which Model? Three Families

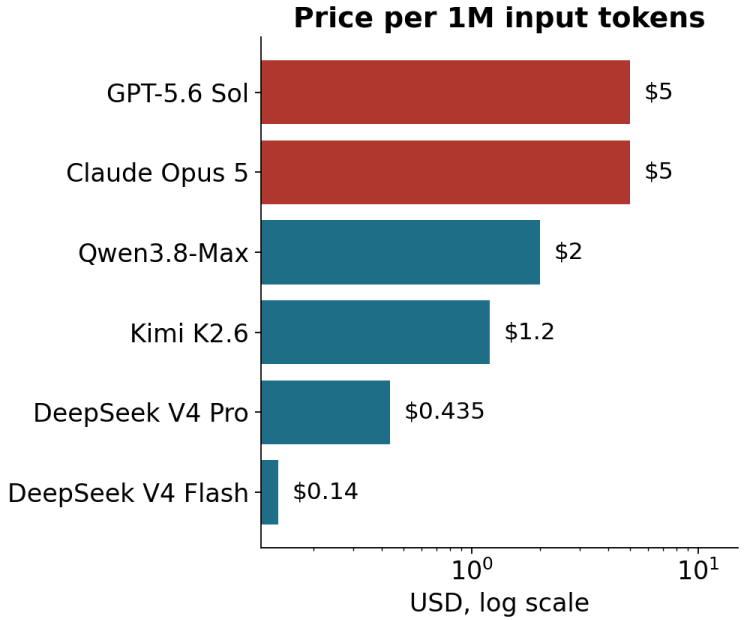
- **OpenAI** (GPT family), **Anthropic** (Claude family),
  Google (Gemini family)
- All three offer strong general models, reasoning variants, long context
- Differences are real but shift with every release; benchmark positions change several times a year
- No European player at the frontier, but see the next two slides
- Today we use Claude because the agent tooling we teach is built around it and because it is currently the benchmark leader for work with data and code
- Trade-off that stays stable: bigger model + more reasoning means better, slower, costlier

The Other Half of the Map: Open Weights

Not just three American labs: **four of the five leading open-weight families are Chinese**, including DeepSeek, Moonshot (Kimi), Zhipu (GLM), and Alibaba (Qwen).







Open-weight models, August 2026 — re-check every number before you show it

Why This Might Matter for Your Research

- **The data never leaves.** Run the weights on your own machine or on UZH infrastructure, and participant data was never sent anywhere
- **Reproducibility.** A downloaded checkpoint is frozen and still runs in five years; a retired API endpoint is gone, and your analysis with it
- **You can say what is in it.** For a closed model you can describe none of the three: weights, training data, or the recipe
- **Cost and lock-in.** No per-token bill, no vendor to be deprecated by

Switzerland has one: Apertus — EPFL, ETH Zurich and CSCS, Apache 2.0, 8B and 70B, 1,800+ languages, ~40% non-English training data.

One Family, Several Tiers

The generation number says *when*. The tier name says *how much*.

Anthropic	OpenAI	Input / output per 1M tokens	Use it for
Fable 5	—	$10 / $50	the genuinely hard thing
Opus 5	Sol	$5 / $25	agent work, analysis design
Sonnet 5	Terra	$3 / $15	the daily driver
Haiku 4.5	Luna	$1 / $5	mechanical work, at volume

A tenfold price spread inside one family. You want the cheapest model that is *strong enough*, and that is a five-minute experiment, not a guess.

Choosing Reasoning Effort

- Most current models let you dial thinking up or down; in Claude Code, use `/effort`: `low`, `medium`, `high`, `xhigh`, `max`
- **The dial sets a budget, not a fixed amount.** Within any level the model decides *per step* whether to think and how much
- **The best setting for most people is the default** (`high`), which is what the documentation recommends
- Turn it down for short, latency-sensitive, unimportant work; turn it up for complicated analysis design and gnarly debugging
- Rule of thumb: pay for reasoning when errors compound; agent work usually qualifies

Two Surfaces — and the API

Surface	What it is	Fits when…
Chat + Projects	conversation; context you bundle in the cloud	questions, drafts, reading, thinking
Claude Code	an agent in *your* folder: `CLAUDE.md`, skills, subagents	the output has to be a **file**

Claude Code runs in the terminal or in the Desktop app. Same engine, same `CLAUDE.md`, same skills, same subagents. That is a window preference, not a decision; the decision is the row above it.

Also there, not a surface: the API. Your own script calls the model directly with no window and no agent. This is where you go when the job is "*classify 4,000 open responses*" rather than "*help me with this.*"

Pick Your Surface — Now

- Run Claude Code in the **terminal** or in the **Desktop app**, your call
- Every walkthrough step is shown on both from the front
- The handout marks every point where the two differ
- Rough guide: comfortable in a terminal, use it; otherwise use the app

This is not a commitment. Same engine either way; switch at lunch and you lose nothing.

From LLM to Agent: Tools

- A plain LLM can only emit text
- An agent is an LLM given **tools**: read a file, edit a file, run a shell command, search the web
- A tool call is structured text the model emits; the harness executes it and feeds the result back
- The model never touches your machine directly; the harness does, under rules you set

The Agent Loop

```
your task
   -> model thinks, calls a tool
   -> result comes back
   -> model thinks, calls the next tool
   -> ... repeats ...
   -> done: model reports back
```

- The model decides which tool, in what order, when to stop
- One prompt can trigger dozens of steps
- This loop is the whole trick; everything today is this loop at different scales

TYPE-ALONG — Handout 1.1: Watch the Loop

Run in your session:

```
What files are in the current folder? List them and say in one line
what each one seems to be for.
```

Expected: one tool call, its result, then a summary naming `CLAUDE.md`, `pyproject.toml`, `data/`, `papers/`, `.claude/`.

Permissions

- Read inside the project: allowed by default
- Write, delete, run commands: the agent must ask first
- Modes, from cautious to trusting:
  Manual: approve every action
  Accept edits: file edits pass, commands still ask
  Plan: agent proposes a plan, makes no file changes
  Bypass Permissions: agent can do everything ('YOLO mode')
  Auto: a second model reviews every task and blocks dangerous tasks
- Today: Manual, until approving edits gets tedious
- Check the indicator first: paid plans now start in Auto; the toy project's `settings.json` forces Manual, Shift+Tab or the selector if not

TYPE-ALONG — Handout 1.2: Trigger the Dialog

```
Create a file called hello.txt containing one sentence about what you
did in the previous step.
```

Expected: a permission request **before** anything is written. Approve it, then check that `hello.txt` exists.

TYPE-ALONG — Handout 1.3 and 1.4: Read, Then Delete

```
Read hello.txt and quote its exact content back to me.
```

No permission prompt; reading is free.

```
Delete hello.txt.
```

Permission prompt again, this time for a shell command.

Part 2: One Research Workflow, End to End

10:00-12:15 — the toy project: **Swiss rents**

How do average rents vary across Swiss cantons, and how robust is the pattern? Five open-access papers, two public datasets.

Step	Content	Time	Clock
1	Project tour and conventions	20 min	10:00-10:20
2	Subagent literature analysis	30 min	10:20-10:50
	Break	10 min	10:50-11:00
3	Live data fetch	20 min	11:00-11:20
4	Exploratory analysis, robustness	30 min	11:20-11:50
5	Stress-test and analysis plan	25 min	11:50-12:15

Step 1: Project Tour and Conventions

10:00-10:20 (20 min)

Prompts (handout, Step 1):

1. "What do you know about this project? Name the research question, then list every skill, rule, and agent..."
2. "Show me `.claude/skills/fetch-swiss-rents/SKILL.md` and explain how the description field decides when this skill gets used."
3. "Run `uv sync` to set up the Python environment."

You should see: the agent knows the project unprompted — question, three skills, one rule, two agents, with paths.

An agent is only as capable as the context it starts with. This project ships that context in a number of different files.

Step 2: Subagent Literature Analysis

10:20-10:50 (30 min)

Prompts (handout, Step 2):

1. "...dispatch the paper-reader agent once per PDF in `papers/` — five runs, in parallel..."
2. "Across the five summaries: which data sources and methods recur...?"
3. If time: "Have the skeptic-reviewer agent review the comparison."

You should see: five subagents launch at once; a few minutes later, five uniform structured summaries; then a one-page synthesis.

Teaching point: delegation. Subagents read in their own context and return only the distilled result.

Break

10:50-11:00 (10 min)

Leave your session open — it keeps its memory.

Back at 11:00: live data from two public APIs.

Step 3: Live Data Fetch

11:00-11:20 (20 min)

Prompts (handout, Step 3):

1. `/fetch-swiss-rents`
2. "From the Zurich rent data: how has the median rent (qu50) for 3-room market housing developed over the available years?"

You should see: BFS cantonal workbook (XLSX) and City of Zurich records (JSON via CKAN) fetched, and for each, a report of which path was taken: live download or local fallback.

Teaching point: skills as reusable procedures, with graceful degradation built in.

Step 4: Exploratory Analysis and Robustness

11:20-11:50 (30 min)

Prompts (handout, Step 4):

1. "Using the BFS dataset, write a script that computes the average rent per canton... Follow the project's conventions."
2. "Rerun the analysis twice: once dropping Zurich and Geneva... and once splitting the period... Does the ranking hold?"

You should see a documented Python script (not chat math) run via `uv run python`, then the robustness variants for the cost of one sentence.

Teaching point: rules and skills shape the output without being mentioned in the prompt. Reruns become cheap.

Step 5: Stress-Test and Plan

11:50-12:15 (25 min)

Prompts (handout, Step 5):

1. Switch to plan mode (CLI: Shift+Tab; Desktop: mode selector).
2. `/grill-me I want to argue that the cantonal rent differences we found are driven by urbanization. Interview me about how exactly this analysis should be implemented before we write the plan.`
3. "Now write the plan for the analysis we just agreed on..."

You should see: the agent asks **one** question at a time, each with a proposed answer (four to six rounds), then a recap; then a plan in the plan-mode approval view that reflects the interview, concessions included.

What You Take Home — and What Comes Next

The morning in one line: **context files + tools + permissions + delegation + planning.**

- Your `swiss-rents` folder keeps working: start a session inside it any time. Every piece is a plain-text file to read, edit, imitate
- 12:15 lunch
- 13:15 advanced workflow demos: a research hub that runs overnight, then a synthetic dataset built from a codebook
- 14:30 one literature workflow, live
- 14:50 hands-on until 16:30: your material, the fresh dataset, or a try-out package, in four tracks
- 16:30 open discussion on responsible use

Lunch

12:15-13:15

Back at **13:15** — the afternoon opens with demos.

Leave your session open. Nothing you did this morning is lost.

Welcome Back

13:15 — welcome to anyone joining for the afternoon.

The morning in one line: **an agent is a model plus tools, running in a loop, inside a folder that tells it what the project is.**

- Everything it knows on startup came from files: `CLAUDE.md`, skills, rules, subagent definitions, all plain text
- Everything it does that touches your machine asks permission first
- Delegation to subagents keeps the main conversation clean
- Planning comes before execution, not after

Afternoon: demos, a dataset built from a codebook, a live literature workflow, your own work, and an open discussion to close.

Part 3a: Advanced Workflow Demos

About seventy-five minutes, Nico, two systems, back to back:

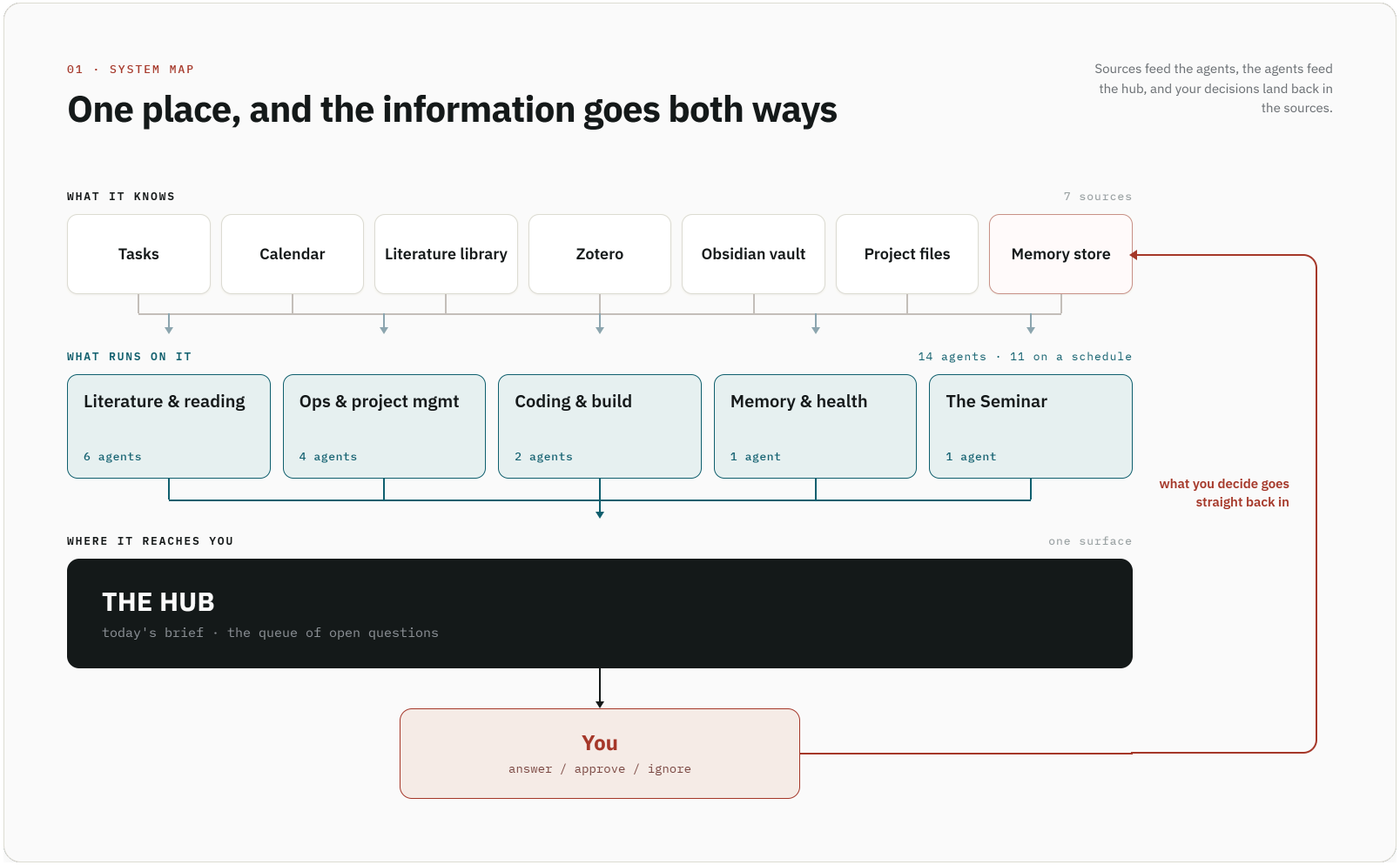
Demo	
A research hub that runs overnight	13:20
A synthetic dataset, rung by rung	~13:45

Then, at 14:30, Paul: **one literature workflow, live.**

Every demo ends on **the smallest version of itself**, where you would start, not what you would deliver.

Watch that part. The big one is only what happens if you keep going.

Demo 1: A Research Hub That Runs Overnight



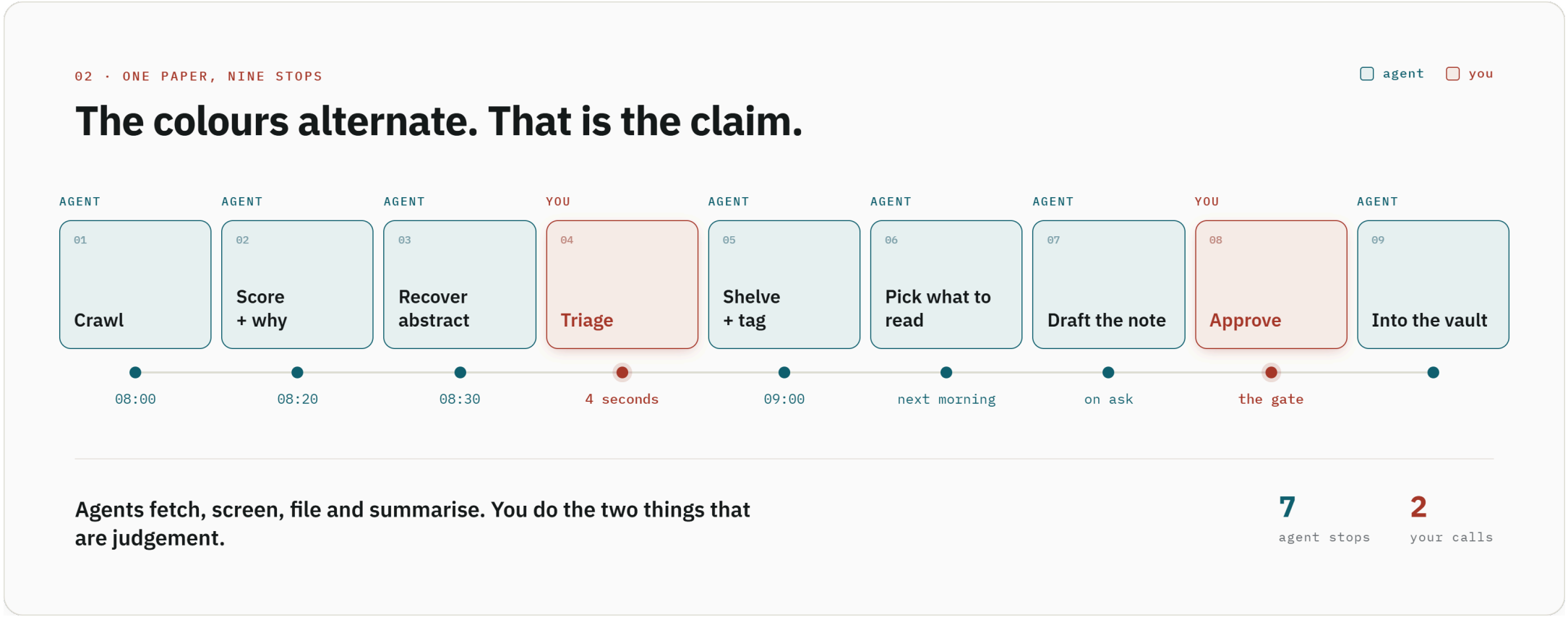
Version One: Five Steps

Before any of the rest of it, the whole thing was this:

1. **Pull a keyword list out of what I already cite** — my own library, read once
2. **Find the databases that have an API** — OpenAlex, arXiv, Semantic Scholar
3. **Fetch and rank** — everything matching, roughly sorted, put in front of me
4. **Thumbs up, thumbs down** — one judgement per paper, nothing else asked of me
5. **The votes rewrite the keywords** — so tomorrow morning it searches differently

Step 5 is the only interesting one. It is what makes this a loop instead of a saved search.

One Paper, Nine Stops



Where You Would Start

Two routes in. Pick one, not both.

1 · **The hub.** Put your tasks somewhere an agent can read them, then write one skill: a `/brief` that reads them and tells you what today actually looks like. Run it every morning.

2 · **Literature.** Point it at your Zotero library, have it pull out the themes you actually work on, then write a scout that searches **one** open database for matches and shows you them when you ask.

Neither is a project. Both are an afternoon, and neither needed a Raspberry Pi.

Demo 2: A Synthetic Dataset, Rung by Rung

~13:45-14:30 · Nico

Three reasons you would want this at all:

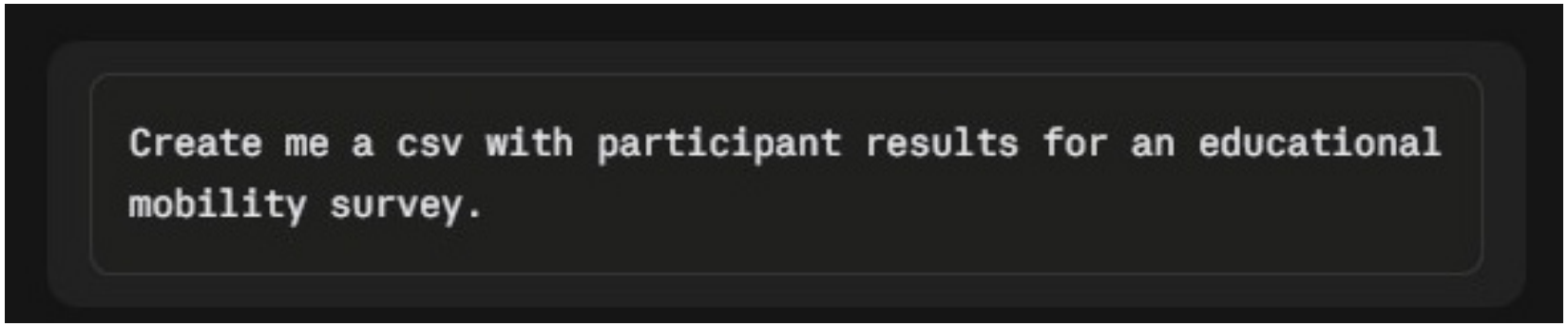
1. **Test your design before you have data**: do your variables exist, can they produce what you want to estimate, does the analysis run
2. **Build datasets for your own students**: a structure you planted, an exercise whose answer you know, grading against ground truth
3. Synthetic replication: it exists, it is contested, not today

The Ladder: Four Rungs

Each rung adds exactly **one** input you decide instead of it.

Rung	You supply	What it buys	Verdict
0	general prompt	a file	nothing is yours
1	the instrument	a form-shaped file	only your variables
2	+ the respondents	representative file	your variables + pipeline
3	+ the process	calibrated file	calibrated variables + pipeline

Rung 0 — You Give It Nothing



What it buys you	What it costs you
a file, in seconds, and an articulate account of it	you chose no variable, including what you are trying to measure
it always produces *something*	it listed the decisions it made for you, and offered to change them **afterwards**

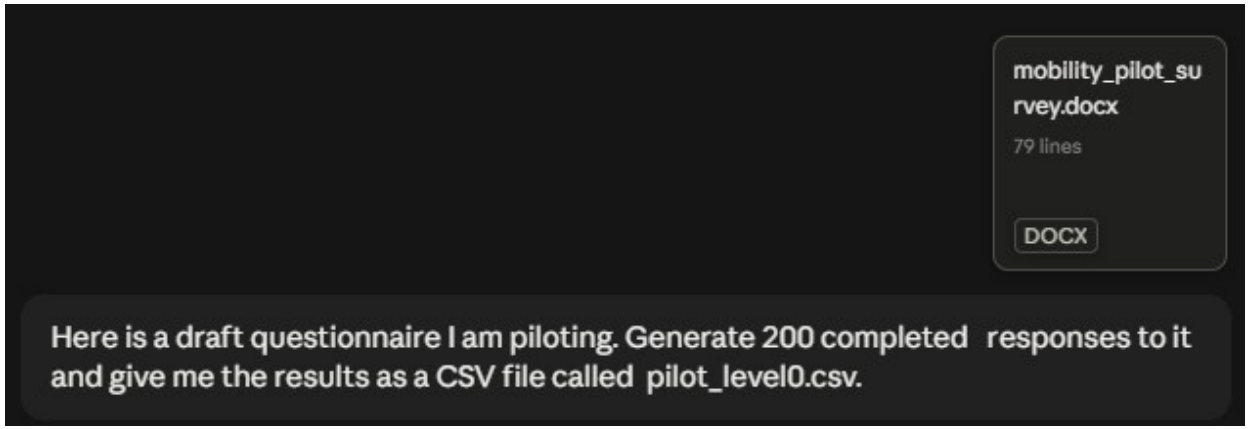
Rung 0 — What Came Back

Same eleven words, same chat box, two accounts.

	Paid account (Opus 5)	Free account (Sonnet 5)
shape	240 × 28	40 × 11
where	eight European countries	US: Northeast / South / West / Midwest
money	`current_income_bracket_chf`	`$25k-$50k`
the outcome	`education_gap_vs_parents`	`mobility_score`, 0–3, undefined

3 shared columns out of 28 and 11: `participant_id`, `age`, `gender`.

Rung 1 — You Give It the Questionnaire



What it buys you	What it costs you
it seeded realistic problems and criticised two items	**it chose who was in your sample**
a file shaped like your instrument, in one turn	**you don't control the shape**

It found problems. It also invented the population that has them.

Rung 2 — You Give It the Respondents Too

```
Here is the same questionnaire, and here are 200 respondent profiles I
built to specified margins. Answer as each of these people. Where a
respondent cannot answer an item as written, leave it blank and record why.
```

What it buys you	What it costs you
a dataset aligned with **your instrument and your sample**	**the joint distribution is still its invention**
it shows where those two do not fit	no direction: no effect size, no target gap
something your analysis script can run on	no seed: run it again, different people

Rung 2 — What Came Back

	invented	ours	real (BFS)
compulsory	0.090	0.140	0.139
vet	0.540	0.315	0.314
matura_hv	**0.000**	**0.235**	**0.234**
tertiary	0.370	0.310	0.313

We can see how an invented sample might not be representative.

Rung 3 — You Give It the Process

```
Read the codebook. Write a script that generates the dataset exactly as
specified in section 10, and a validation step that runs the checks in
section 9. Use only numpy and pandas. Print the seed.
```

What it buys you	What it costs you
marginals **measured, not guessed**	it is only ever as good as the page you wrote
same codebook, same seed, **same file on any machine**	it's still only a representation of a specific sample you picked
there's a ground truth	

Rung 3 — What Came Back

```
VALIDATION   n=2400   seed=20260908

[1] Structural   ok — ids unique, ranges respected, NA pattern matches
                     employment exactly
[3] parent_isei mean 43.89  target 44.00  ·  edu_years 13.50  target 13.50
[4] Structure — correlations all positive, identical on any machine
[6] The headline           coef      se       t
    female               -5.214   0.584  -8.927
```

Same codebook, same seed, same file. On any machine.

Where to use it, what to watch

You can try this on your own surveys. Export your instruments, generate responses at the rung you need, and **run your real analysis script on them, start to finish.**

What it does not cover

- The LLM will usually not evaluate the quality of your instruments and sample
- It still comes with its own biases, from its training and guardrails
- It can overoptimise/overthink things

A simulated respondent answers all three cheerfully. It was never confused and has no social desirability to exhibit.

You can ask an agent to audit the wording *as its own task*: mine did,

"How Do I Stop This Happening TO My Survey?"

There is no reliable detection. Four things that do help:

1. **Attention checks are mostly finished.** AI passes most of them.
2. **Record keystrokes and paste events.** Boring, and the best-evidenced defence. Source text as an image plus disabled paste nearly halves LLM use, but disabling paste also lowers data quality.
3. **Pre-register your exclusion rules before you look.** Deciding afterwards which responses "look like AI" can be exclusion p-hacking.

Part 3b: One Literature Workflow, Live

14:30-14:50 · Paul

One niche topic, five stages, end to end:

search → download → curate → excerpt → read

- Stage 1 runs live: the search agents launch in front of you
- Stages 2-4 ran at home; you see the real artefacts they produced
- Stage 5 runs live again: a reader subagent works a paper while we watch

The read stage ships in the repository as a skill and an agent; the other stages are documented there, and their skills are yours to write.

Part 3c: Hands-On, On Your Own Material

14:50-16:30, break at 15:40

Three suggestions, from what you voted for, and a fourth that is whatever you want.

Track	Content
Pipelines and automation	APIs, scraping, OCR, cleaning, scheduled runs
Quantitative analysis	robustness checks, reproducible documented scripts
Literature workflows	search, screening, summarising, Zotero
Whatever you now want to build	a research hub of your own, a teaching tool, the thing you thought of at 13:40

Small groups are encouraged. Both of us circulate.

What to Work On — Three Equally Good Options

Three equally good options:

1. **Your own material**: a project, a dataset, a pile of PDFs, a script you never documented
2. **The dataset from half an hour ago**: clean, shareable, and you can break it without consequences
3. **The try-out package for your track**: a task brief plus data or papers, ready to go

One Thing Everyone Does Before 16:30

Whatever you are working on, at some point:

1. Ask the agent to **audit its own output** — write down what it did, what it assumed, and what could be wrong
2. Send that audit to a **subagent that has not seen the conversation**, and ask it to review the work blind

Verification is something you do, not something you promise.

When the Output Is a Decision

Ask for the audit as **something you can work in**, not something you read.

- Sorted by **what each finding blocks**, not by where it was found
- Every option carries **its consequence**, not just its name
- The evidence sits next to the claim; you check it, you do not trust it
- A box for your reasoning, so the decision goes back where the work is

A conclusion asks you to trust it. An artefact shows its working and asks you to decide.

Break

15:40-15:55 (15 min)

Leave your session open — it keeps its memory.

Part 4: An Open Discussion

16:30-16:55, no slides after this one.

- **Deskilling and cognitive offloading**: what happens to a skill you stop practising?
- **Detection**: it mostly does not work, and when it does, on whom?
- **Dependence**: what does daily AI use do to attention and judgement?
- **Our own work**: today was the floor, not the ceiling. What is a social scientist for?
- **The defensible core**: what can you use this for, safely and ethically, starting tomorrow?

A **reading list** for all five is in the repository and in the follow-up package.

Before You Leave

16:55-17:00 · **Six questions, three minutes, anonymous** — now, before you stand up.

forms.cloud.microsoft/e/66HscTeqUe

- **Pace and level** — morning and afternoon, separately
- What was **most useful**?
- What would you **cut or shorten**?
- What was **missing**?
- Did the **setup** work before today?
- Would you **recommend** it?

Thank you.